Supplementary Material: LLM Agent Specifications, Prompt Templates, Open Benchmark Repository, and Implementation Details

UrbanTrace: LLM-Assisted Discovery and Semantics-Aware Integration of Spatial Data (Submission #2151)

Contents

Part I
LLM Agent Specifications and Prompt Templates

1 Introduction

In what follows, we provide technical details, implementation metrics, and exact prompt templates for the three LLM-based agents within the UrbanTrace framework: the Semantic Data Discovery Agent, the Semantic-Aware Operator Copilot, and the LLM Synthesis Agent. This material is provided to support full reproducibility of our system configuration, token management strategies, and prompt engineering logic.

2 System Configurations and Model Matrix

To maintain low execution variability and deterministic reasoning, all agents are evaluated using a low temperature setting (typically $T = 0.0$). All production models are deployed through Vertex AI. Table 1 summarizes the operational characteristics of each agent configuration within the UrbanTrace architecture.

Table 1: Empirical Operational Profile and Token Footprint per Agent Session

Agent Component	Production Model	Input Tokens	Output Tokens	Latency
Data Discovery Agent	Gemini 3 Pro	∼20.2K tokens	∼320 tokens	∼9.6 s
Operator Copilot Agent	Claude 4.6 Opus	∼1.55K tokens	∼640 tokens	∼12.0 s
LLM Synthesis Agent	Claude 4.6 Opus	∼1.48K tokens	∼490 tokens	∼10.0 s

3 Semantic Data Discovery Agent

3.1 Context Management and Catalog Scalability

The Data Discovery Agent is responsible for parsing a data lake containing 112 benchmark datasets. Rather than injecting raw spatial coordinates or full data tables into the LLM context window, which would cause severe token bloat and exceed typical operational boundaries, UrbanTrace employs a decoupled profiling strategy:

1. **Offline Profiling (Atlas Profiler):** Strips away structural coordinate layers and distills tables into compact geometric and schema summaries (bounding boxes, row counts, attribute data types, semantic type annotations, and geospatial classifier labels).

2. **Natural Language Summarization (AutoDDG):** Automatically synthesizes each dataset schema into a concise semantic description.

Each dataset is serialized into a single line using the following compact format:

```
- id=[dataset_id]: geometry=[type]; attribute_keywords=[kw1, kw2, ...];
  nb_columns=[N]; columns=[(col_name, type=[structural_type],
  semantic=[semantic_type], geo=[geo_classifier]); ...];
```

```
  description=[AutoDDG natural language summary]
```

Listing 1: Dataset Catalog Entry Format

As a result of this compression, the entire metadata catalog for all 112 datasets is completely represented within an input footprint of approximately **20,200 tokens**. The system enforces a per-entry character cap derived from a token budget of 272,000 tokens total, reserving 20,000 tokens of headroom for system instructions, tool schemas, and dashboard state. For modern large context windows (spanning 200k to over 1M tokens), the catalog occupies less than 10% of total capacity, allowing the agent to perform comprehensive catalog-wide discovery within a single request context without requiring lossy vector-search pre-filtering steps. As noted in the main paper, a current limitation of this approach is the linear growth in token cost as catalog size scales; future work will integrate retrieval-based pre-filtering methods to select a relevant subset of the catalog before LLM inference, reducing both latency and cost at scale.

3.2 System Prompt Structure

The agent receives a single merged system message composed of four ordered parts at inference time:

1. **Role declaration:** A brief statement establishing the agent as a helpful assistant for UrbanTrace.
2. **Dataset catalog context:** The full compressed catalog of all 112 datasets, injected using the per-entry format described above.
3. **Live dashboard state:** A JSON snapshot of the current ReactFlow canvas, including active dataset nodes, integration nodes, and their connections—enabling the agent to make suggestions that are aware of what is already on the canvas.
4. **Tool usage rules:** Six explicit behavioral constraints governing when and how to invoke the `suggest_add_dataset_node` tool, enforcing conservative suggestion counts, exact dataset ID matching, and pending-state semantics (the agent must not claim a dataset was added until the user accepts the suggestion).

3.3 Prompt Template

The complete system prompt template is shown below, with dynamic placeholders in brackets:

```
You are a helpful assistant for UrbanTrace.

You have access to these UrbanTrace dataset descriptions and metadata:
UrbanTrace dataset catalog:
Use this context when answering dataset questions.
- id=[dataset_id_1]: geometry=[type]; attribute_keywords=[...];
  nb_columns=[N]; columns=[...]; description=[AutoDDG text description]
- id=[dataset_id_2]: geometry=[type]; attribute_keywords=[...];
  nb_columns=[N]; columns=[...]; description=[AutoDDG text description]
... [remaining dataset entries for all 112 datasets]

You have access to the current UrbanTrace dashboard state (ReactFlow
   canvas):
[JSON snapshot: node_count, edge_count, dataset_nodes with
```

```
geometry/columns/description, integration_nodes, connections]
Use this dashboard context directly when answering questions about nodes,
connections, and selected datasets.

Copilot rules:
1. Use suggest_add_dataset_node when the user asks to add a dataset,
   place a dataset, or asks which datasets are relevant or good
   starting points.
2. Suggest only the strongest few matches.
3. Use exact dataset_id values from the dataset context. Add color_by
   only when there is a clearly relevant numeric column in the metadata.
4. Include a short reason for each suggestion.
5. Suggestions are pending user decisions. Do not say a dataset was
   added unless the user accepted it.
6. Always give a short, decision-oriented reply. If no strong catalog
   match exists, answer in prose and do not call tools.
```

Listing 2: Semantic Data Discovery Agent – System Prompt Template

3.4 Expected Input and Output

The following illustrates a representative end-to-end turn. The user submits a planning query in natural language:

```
Find datasets in NYC related to student population, poverty or income,
and access to public resources like libraries or Wi-Fi to study
after-school program needs.
```

Listing 3: Discovery Agent – Example User Query

The agent searches the injected catalog and responds with multiple `suggest_add_dataset_node` tool calls. The abbreviated ingestion payload and three representative calls from this turn are shown below:

```
// System Message (merged, four parts -- abbreviated)
You are a helpful assistant for UrbanTrace.
UrbanTrace dataset catalog:
Use this context when answering dataset questions.
- 2013_yellow_taxi_trip_data: geometry=Point; rows=500000;
  description=This dataset provides detailed records of individual taxi
  trips taken during 2013. Each record includes the pickup and drop-off
  date and time, passenger count, trip distance, and precise geospatial
  coordinates for the start and end locations...
- 2017_2018_schools_nypd_crime_data_report: geometry=Point; rows=1919;
  description=This dataset provides a comprehensive directory of school
  buildings located throughout New York City's five boroughs, including
  safety and incident statistics for each location...
... [remaining 110 dataset entries]
[Dashboard state snapshot: node_count, dataset_nodes, connections]
[Copilot tool usage rules 1-6]

// User Message
Find datasets in NYC related to student population, poverty or income,
and access to public resources like libraries or Wi-Fi to study
```

```
after-school program needs.

// Resulting Assistant Tool Calls (showing 3 of 5 suggestions)
{ "tool": "suggest_add_dataset_node",
  "parameters": { "dataset_id": "NYC_poverty_rate",
                  "color_by": "pop_pov_pct",
                  "reason": "Neighborhood poverty rates to pinpoint
                             high-need areas." }}
{ "tool": "suggest_add_dataset_node",
  "parameters": { "dataset_id": "NYC_libraries",
                  "reason": "Library locations as key public resources
                             where students access study spaces, internet,
                             and after-school programs." }}
{ "tool": "suggest_add_dataset_node",
  "parameters": { "dataset_id": "dycd_program_sites",
                  "reason": "DYCD community program sites (2020-2022)
                             including youth services and after-school
                             programs -- directly relevant to identifying
                             existing coverage and gaps." }}
```

Listing 4: Discovery Agent – Ingestion Payload and Structured Output

Following tool execution, a second non-streaming completion pass generates the final natural language reply. This follow-up pass receives the original user request along with the serialized tool calls and responses, and is instructed to produce a 1–3 sentence reply explaining the suggestion. This two-pass design decouples tool invocation from natural language generation, ensuring the user always receives an explanatory text response alongside the dataset recommendations.

4 Semantic-Aware Operator Copilot Agent

4.1 Operational Strategy

The Copilot Agent determines valid geospatial transformations by analyzing spatial geometry constraints and statistical distributions. It matches incoming source datasets against a target zoning polygon to select point-safe or area-weighted operations while verifying whether an attribute is *Extensive* (e.g., total counts) or *Intensive* (e.g., densities or rates). The agent is invoked via a dedicated structured JSON task call, separate from the main copilot chat, and is constrained to return only operators drawn from the explicitly provided `available_mapping_operators` and `available_aggregation_operators` arrays; it is explicitly instructed never to invent operator names.

4.2 System Prompt Template

```
You are an expert Spatial Data Scientist and GIS Architect. Your task
is to select the optimal Zoning Mapping and Zoning Aggregation operators
from the provided lists.

Step 1: Analyze the Context & Metadata
Review dataset_name and column_metadata.name together. If the column
name is generic (e.g., value, count, total, metric), you MUST rely on
dataset_name to determine the meaning. Then review num_distinct_values,
distribution (mean, coverage), and sample_data.

Step 2: Classify the Data Type
- Extensive (Count/Total): high num_distinct_values, larger means,
  values scale with area
- Intensive (Rate/Density): decimals/floats, keywords like
  rate/avg/median/density
- Categorical/Ordinal (Index): low num_distinct_values (often 1-10),
  integer classes, index-like labels

Step 3: Geospatial Reasoning & Operator Selection
You are provided target_geometry and valid arrays:
available_mapping_operators and available_aggregation_operators.
- If target is Point/MultiPoint, area-weighted mapping is invalid.
  Choose point-safe mapping.
- Extensive -> aggregation should preserve totals (typically Sum)
- Intensive -> aggregation should avoid absurd accumulation
  (typically WeightedMean/Mean/Density)
- Categorical/Ordinal -> use discrete grouping (typically Majority)

Return ONLY a valid JSON array with one object per source variable,
each containing: dataset_name, column_name, classification,
zoningMapping, zoningAggregation, reasoning.
Never invent operators not present in provided arrays.
Do not return markdown.
```

Listing 5: Operator Copilot Agent – System Prompt Template

4.3 Expected Input and Output

```
// Input Variable Array Context
{
  "task": "Determine zoning spatial mapping and aggregation operators
           based on statistical metadata.",
  "zoning_target": {
    "dataset_name": "NTA_Neighborhood_Tabulation_Areas",
    "geometry_type": "MultiPolygon"
  },
  "available_mapping_operators": [
    "CentroidZoning", "AreaWeightedZoning", "LengthWeightedZoning"
  ],
  "available_aggregation_operators": [
    "SumZoning", "WeightedMeanZoning", "DensityZoning", "MajorityZoning"
  ],
  "source_variables": [
    {
      "dataset_name": "NYC_Schools",
      "original_geometry": "Point",
      "column_metadata": {
        "name": "student",
        "structural_type": "http://schema.org/Integer",
        "mean": 322.29
      }
    }
  ]
}

// Resulting Assistant Output JSON Array
[
  {
    "dataset_name": "NYC_Schools",
    "column_name": "student",
    "classification": "Extensive",
    "zoningMapping": "CentroidZoning",
    "zoningAggregation": "SumZoning",
    "reasoning": "The 'student' column represents school enrollment
                  counts. The source geometry is Point, so area-weighted
                  mapping is invalid; CentroidZoning is the appropriate
                  point-safe mapping. SumZoning preserves totals when
                  aggregating points into polygons."
  }
]
```

Listing 6: Copilot Agent – Payload and Structured Output

5 LLM Synthesis Agent

5.1 Operational Strategy

The Synthesis Agent reasons across multi-variable data tables to determine optimization polarity (direct/normal scaling vs. inverted scaling) and assigns normalized weights ($[0, 1]$) based on the user's high-level urban prioritization goal. Like the Operator Copilot, it is invoked as a standalone structured JSON task, not through the main conversational copilot loop, and is required to return a valid JSON array with no markdown wrapping. When a `goal` field is present in the payload, the agent uses it to calibrate both direction and relative weight; when absent, it infers the most semantically coherent goal from variable names and metadata.

5.2 System Prompt Template

```
You are an expert urban analytics copilot. Your task is to synthesize
hotspot-priority semantics.

The input may include an optional "goal" field describing what
"Priority" means in this context (e.g., "pedestrian safety risk",
"economic vulnerability", "environmental burden").
If present, use it to inform both direction and relative weight for
each variable.
If absent, infer the most semantically coherent goal from the variable
names and metadata.

For each source variable, infer:
- direction: "normal" (higher raw value = higher priority) OR
  "inverted" (lower raw value = higher priority)
- reasoning: short justification referencing the goal, dataset context,
  column metadata, and sample values
- weight: relative importance in [0,1]; variables more directly tied
  to the goal should receive higher weight

Use semantic cues from dataset_name and column_name, and validate with
sample_data statistics.
Examples (goal-agnostic defaults):
- crashes/injuries/poverty/pollution -> normal
- income/coverage/infrastructure/safety score -> inverted

Return ONLY JSON array of objects with fields:
dataset_name, column_name, direction, reasoning, weight.
Do not return markdown.
```

Listing 7: LLM Synthesis Agent – System Prompt Template

5.3 Expected Input and Output

```
// Input Variable Array Context
{
  "task": "Synthesize hotspot priority polarity and weights.",
  "goal": "economic vulnerability",
  "source_variables": [
```

```
    {
      "dataset_name": "NYC_poverty_rate",
      "column_name": "2017",
      "column_metadata": {
        "structural_type": "http://schema.org/Float",
        "mean": 0.183
      },
      "sample_data": [0.272, 0.148, 0.243]
    }
  ]
}

// Resulting Assistant Output JSON Array
[
  {
    "dataset_name": "NYC_poverty_rate",
    "column_name": "2017",
    "direction": "normal",
    "reasoning": "Higher poverty rates directly indicate greater
                  socioeconomic vulnerability, so higher values equate
                  to higher priority hotspot status under the economic
                  vulnerability goal.",
    "weight": 0.35
  }
]
```

Listing 8: Synthesis Agent – Payload and Structured Output

Part II
Open Benchmark Repository

6 Introduction

To support reproducibility and transparency, we make our benchmark tasks and quantitative ablation studies publicly available on GitHub. The open-source repository and data can be accessed at:

`https://github.com/VIDA-NYU/urbanTrace/tree/main/benchmark`

7 Repository Contents

The repository is organized around the `benchmark/` directory, which contains all assets required to understand and reproduce the two ablation studies reported in the paper.

7.1 Benchmark Tasks (28 Urban Research Scenarios)

The core evaluation corpus comprises 28 real-world urban research scenarios grounded in NYC planning, policy, and public-service contexts. Each scenario includes a natural-language prompt describing a realistic analytical objective (without naming specific datasets), a manually validated gold set of 3–5 relevant datasets, and supporting references to policy documents and public data sources.
Key assets:

- `urbantrace_agent_benchmark.json` — machine-readable manifest consumed by the benchmark runner.
- `cases/` — per-case writeups with task rationale, gold dataset mappings, and supporting references.

7.2 Ablation Study 1: Dataset Discovery Performance

This ablation evaluates the UrbanTrace discovery agent's ability to recover relevant datasets for each benchmark scenario. It is structured as a controlled ablation over five catalog context conditions: (i) Name only, in which the agent receives only dataset names; (ii) Source description only (`no_profile`), in which names are augmented with original source descriptions from NYC OpenData; (iii) Profile only (`no_description`), in which names are augmented with explicit Atlas Profiler profile metadata; (iv) AutoDDG description only (`profiled_description`), in which names are augmented with AutoDDG-generated descriptions derived from Atlas Profiler profiles, but no explicit profile metadata is provided at inference time; and (v) Full, which combines AutoDDG-generated profile-derived descriptions with explicit Atlas Profiler profile metadata. Benchmarks are run with two model backbones (Gemini 3 Pro and GPT-5 mini), each condition repeated 5 times to account for stochastic variance. Performance is measured via precision, recall, and F1 over set overlap against the gold dataset set.
Key assets:

- `scripts/run_dataset_discovery_benchmark.py` — benchmark runner (supports `--ablation`, `--runs`, `--llm-model`, `--output` flags).

- `scripts/visualize_results.ipynb` — notebook for aggregating CSV results and generating figures.
- `results/ablation1_results.png` — barplot figures reported in the paper.

7.3 Ablation Study 2: Spatial Operator Selection Accuracy

This ablation evaluates the UrbanTrace Copilot's automated selection of spatial mapping and aggregation operators across 100 NYC spatial integration scenarios. It benchmarks five conditions: a rule-based legacy GIS baseline, three naive LLMs (GPT-4o-Mini, Claude 3.5 Sonnet, Claude 4.6 Opus) given only dataset and column names, and the full UrbanTrace Copilot augmented with statistical profiling context. Operator selection is evaluated on two dimensions: Geometric Validity (correct spatial mapping operator) and Semantic Validity (correct aggregation operator).
Key assets:

- `scripts/quantitative_ablation-operator_selection_accuracy.ipynb` — Jupyter notebook implementing the full benchmark pipeline.
- `ground_truth_curated.json` — human-validated ground truth annotations (semantic class and aggregation operator per column).
- `scenarios_from_benchmark.json` — 100 randomly generated spatial integration scenarios (source column → target boundary pairs).
- `benchmark_results_operator_selection.json` — per-scenario prediction trace for all five conditions.

All benchmark assets were constructed using an LLM-assisted, human-validated workflow. Ground truth annotations and gold dataset mappings were manually curated by the authors to ensure evaluation integrity.

Part III
Implementation Details

8 Source Code

The source code for UrbanTrace is publicly available at:

`https://github.com/VIDA-NYU/urbanTrace`

9 System Architecture

UrbanTrace follows a two-tier client–server architecture. The frontend is a React single-page application built with Vite, and the backend is a Python FastAPI service. Communication between tiers uses REST over HTTP with GeoJSON as the primary data exchange format. The backend is a lightweight FastAPI service (`backend/main.py`) responsible for serving GeoJSON datasets, dataset metadata, and executing spatially intensive operations such as H3-based set operations that would be prohibitive to run client-side.

9.1 Technology Stack

Table 2 summarizes the key technologies used across the system.

Table 2: UrbanTrace Technology Stack

Layer	**Technology**
Frontend framework	React, Vite
Canvas & node graph	React Flow
WebGL visualization	Deck.GL
Spatial indexing	H3 (Uber Hexagonal Hierarchical Spatial Index)
Backend	Python, FastAPI
Data format	GeoJSON

9.2 Prerequisites

The system requires Node.js v16 or higher for the frontend and Python 3.9 or higher for the backend.